



MÓDULO 01: Introducción a Antigravity-IDE, Tipos Primitivos, Cadenas y Formateo

Curso: Python para Ciencia Abierta (CSIC)

Autor: Gustavo Liñán Cembrano

Fechas: 5 a 8 de Octubre de 2026

Tiempo Estimado de Impartición: 1 hora

Módulo: 01 - Primitive Data Types, String Manipulation & Advanced Formatting

Este cuaderno está diseñado para ayudarte a entender los tipos de datos básicos en Python y cómo trabajar con ellos usando Jupyter Notebooks en Antigravity-IDE.

Objetivos de Aprendizaje:

- Dominar los tipos primitivos (`int` , `float` , `str` , `bool`) y la inferencia dinámica de tipos.
- Aprender el uso de `print()` y el formateo avanzado con `f-strings` .
- Aplicar métodos nativos de cadenas (`.strip()` , `.replace()` , `.join()` , `words_to_ignore`).
- Comprender la división entera `//` , el módulo `%` y la inmutabilidad de cadenas.

Preparación del entorno de trabajo para el CURSO

CONFIGURAMOS:

1. **Abre Antigravity-IDE y selecciona tu espacio de trabajo.**
2. **Entorno Virtual (`venv`):** Asegúrate de que el entorno virtual del curso esté activado (`./venv/bin/python`).

3. **Kernel de Jupyter:** Selecciona el kernel registrado para el curso (`curso_ciencia_abierta`).

Cuadernos Jupyter

Un cuaderno Jupyter está compuesto por celdas, que son bloques independientes de contenido. Existen principalmente dos tipos de celdas:

- **Celdas de código:** donde se escriben y ejecutan instrucciones en un lenguaje de programación (como Python).
- **Celdas Markdown:** que se utilizan para escribir texto con formato, explicaciones, ecuaciones o enlaces.

INTRODUCCIÓN A LOS TIPOS DE DATOS BÁSICOS EN PYTHON

1. Lo primero que tenemos que aprender a hacer es comentar

Comentar vuestros proyectos es fundamental tanto para vosotros mismos, como para aquellos que vayan a continuar o reusar vuestro trabajo.

Por bueno que sea tu código, sin comentarios es bastante inútil.

CONSEJO: Bájate de la atalaya... escribe comentarios claros que expliquen el *porqué* de las operaciones.

```
In [ ]: # PRUEBA A ESCRIBIR TU PRIMER COMENTARIO EN ESTA CELDA Y EJECÚTALA
# Este es un comentario de una sola línea en Python
print("¡Comentarios verificados correctamente en AntigraVity-IDE!")
```

2. Uso de `print()`

`print()` es la función más básica a través de la cual un programa te puede proporcionar información a través de la terminal o del cuaderno.

```
a = 5
b = a**2
print("a es igual a ", a, "mientras que b vale", b)
```

```
In [ ]: a = 5
b = a**2
```

```
print("a es igual a", a, "mientras que b vale", b)
```

3. Uso de `print()` con formato de f-strings

Mi consejo es que siempre imprimas con el `print(f"...")` o `print(f' ')`.

El operador `f"..."` (Python 3.6+, PEP 498), también llamado formato de cadena mediante f-strings, te permite insertar variables o expresiones directamente dentro de una cadena de texto.

```
In [ ]: valor = 3.141592653589793238462643383279502
print(f"El valor de pi con 1 decimal {valor:.1f}")
print(f"El valor de pi con 2 decimales {valor:.2f}")
print(f"El valor de pi con 4 decimales {valor:.4f}")
print(f"El valor de pi con 6 decimales {valor:.6f}")
```

```
In [ ]: Seconds_In_A_Year = 60 * 60 * 24 * 365
MiEdad = 52

print(f"He vivido más de {MiEdad * Seconds_In_A_Year:.1e} segundos (Formato
print(f"He vivido más de {MiEdad * Seconds_In_A_Year:.3e} segundos (Formato
print(f"He vivido más de {MiEdad * Seconds_In_A_Year} segundos (Valor entero
```

```
In [ ]: nombre = "Ana"
edad = 27
print(f"{nombre} tiene {edad} años.") # Ana tiene 27 años

# También puedes realizar operaciones directamente dentro del f-string
print(f"El doble de pi es {2 * valor:.3f}") # El doble de pi es 6.283

print("Demostración de alineación dinámica con f-strings:")
for sep in range(2, 20, 4):
    print(f"{nombre:>{sep}}, {edad:>{sep}}")
```

```
In [ ]: # Una de las ventajas de Python es que no es necesario declarar tipos... se

# Tipos numéricos y operaciones
x1 = 10          # Un entero (int)
r1 = 3.14159     # Un float (real)
z1 = 2 + 3j      # Un complejo (complex)

print(f"Entero: {x1} -- tipo: {type(x1)}") # \t mete una ta
print(f"Flotante: {r1} -- tipo: {type(r1)}")
print(f"Complejo: {z1} -- tipo: {type(z1)}")

z1 = 4           # Cambio el tipo de una variable ya declarada. Tipado Dinámi
print(f"Ahora z1 vale {z1} -- tipo: {type(z1)}")
```

4. Operador Módulo `%` en Python

El operador `%` devuelve el resto de una división entera. Es muy útil para comprobar divisibilidad, determinar si un número es par o impar, o realizar operaciones cíclicas (por ejemplo, rotar índices en una lista).

Sintaxis Básica

```
resto = a % b
```

```
In [ ]: # División simple con resto
a = 10
b = 3

resto = a % b
print(f"10 % 3 = {resto} (salida: 1, porque 10 ÷ 3 = 3 y sobra 1)")

print(f"15 % 6 = {15 % 6} (salida: 3, porque 15 ÷ 6 = 2 y sobra 3)")

# Comprobar si un número es impar usando %
Es_Impar = (a % 2 == 1)
print("El número", a, "es impar?", Es_Impar, "tipo:", type(Es_Impar))
```

5. Operador División Entera `//` en Python

El operador `//` realiza una división entera, es decir, devuelve solo la parte entera del cociente, descartando los decimales. Es muy útil cuando quieres saber cuántas veces un número cabe dentro de otro sin preocuparte del resto.

```
In [ ]: a = 23
b = 7

cociente = a // b
print(a, "//", b, "es", cociente)

# Combinando con el operador módulo (%)
print(f"El resultado de {a}://{b} es {a//b} con {a%b} de resto")
```

```
In [ ]: # Operando con variables de distinto o mismo tipo
x2 = 4
r2 = 2.12**2 # Para elevar al cuadrado
z2 = 3 - 6j

x3 = x1 + x2
r3 = r1 + r2
z3 = z1 + z2

print(f"Suma enteros S = {x3}")
print(f"Suma enteros directamente en el print sin asignar a variable: S = {x3}")
print(f"Suma floats {r1:.3f} + {r2:.3f} restringida a 3 decimales: S = {r3:.3f}")
print(f"Suma complejos {z1} + {z2} = {z3}")
```

```
In [ ]: # Sumamos un real y un entero
m1 = x1 + r1
m2 = x1 * z1

print(f"Suma de distintos tipos {x1} + {r1} = {m1} tipo: {type(m1)}")

# Convertimos m1 en entero de nuevo
m1 = int(m1)
print(f"Suma de distintos tipos convertida a entero {x1} + {r1:.3f} = {m1} t

print(f"Entero x complejo {x1} x {z1} = {m2} tipo: {type(m2)}")
print(f"Real x complejo {r1:.3f} x {z1} = {r1*z1:.3f} tipo: {type(r1*z1)}")
print(f"Complejo x complejo {z1} x {z2} = {z1*z2} tipo: {type(z1*z2)}")
```

```
In [ ]: # Tipado dinámico y conversión a string / bool
x = "una cadena"
print("x ahora es:", x, "y su tipo es", type(x))

a = 0.0014
b = str(a)
c = bool(a)

print(f"a = {a}, tipo: {type(a)}")
print(f"b = '{b}', tipo: {type(b)}")
print(f"c = {c}, tipo: {type(c)}")
```

6. Gestión Básica NATIVA de Cadenas en Python (sin librerías externas)

En Python, una cadena (`str`) es una secuencia inmutable de caracteres.

Concatenación

```
saludo = texto1 + ", " + texto2
```

Sustitución

```
nuevo_saludo = saludo.replace("Mundo", "Python")
```

Comparación

```
texto1 == texto2
```

Método `.join()`

Su trabajo es unir todos los elementos de una lista en una sola cadena usando el string llamante como separador: `separador.join(lista)`

Normalización de Títulos y Preservación de Siglas Científicas:

Al usar `.strip().title()`, siglas como "CSIC" pasarían a "Csic". Para evitarlo, usamos la lista: `words_to_ignore = ['CSIC', 'USE', 'NASA']`.

```
In [ ]: # Cadenas, operaciones y join
texto1 = "Hola"
texto2 = "Mundo"

# Concatenación
saludo = texto1 + ", " + texto2
print(saludo)

# Sustitución
nuevo_saludo = saludo.replace("Mundo", "Python")
print(f"{nuevo_saludo} ----> longitud: {len(nuevo_saludo)}")

# Comparación
print(f"¿Son iguales? {texto1 == texto2}")

# Uso de .join() con autores científicos
autores = ["Einstein, A.", "Curie, M.", "Feynman, R."]
autores_str = "; ".join(autores)
print(f"Autores unificados: {autores_str}")

# Tratamiento de siglas científicas (words_to_ignore)
raw_title = "  análisis exploratorio de datos de investigación CSIC  "
clean_title = raw_title.strip().title()

words_to_ignore = ["CSIC", "USE", "NASA"]
for word in words_to_ignore:
    clean_title = clean_title.replace(word.title(), word)

print("")
print(f"Título original: '{raw_title}'")
print(f"Título limpio (siglas preservadas): '{clean_title}'")

# Normalización de DOI
doi_raw = " DOI: 10.1038/s41586-020-2649-2 "
doi_clean = doi_raw.strip().replace("DOI:", "").strip()
print(f"DOI normalizado: https://doi.org/{doi_clean}")

# Intentemos cambiar un elemento... RECUERDA: LAS CADENAS SON INMUTABLES
try:
    nuevo_saludo[0] = 'C' # Esto lanza un TypeError
except TypeError as e:
    print("")
    print(f"✓ Capturado TypeError esperado (Inmutabilidad de cadenas): {e}")
```

```
In [ ]: # Forma avanzada de modificar cadenas mediante rebanado (Slicing):
texto = "hola"
i = 2 # Posición que queremos cambiar (la 'l')

nuevo_caracter = 'K'

if 0 <= i < len(texto):
    texto_modificado = texto[:i] + nuevo_caracter + texto[i+1:]
    print(f"Texto original: {texto}")
    print(f"Texto modificado: {texto_modificado}")
else:
    print("Índice fuera de rango")
```

7. Ejercicios Prácticos de Autoevaluación

Ejercicio 1:

Limpia la cadena " investigación científica NASA y CSIC ", convirtiéndola a Title Case pero manteniendo "NASA" y "CSIC" en mayúsculas.

```
In [ ]: # SOLUCIÓN EJERCICIO 1:
raw_ex1 = " investigación científica NASA y CSIC "
clean_ex1 = raw_ex1.strip().title()
for w in ["NASA", "CSIC"]:
    clean_ex1 = clean_ex1.replace(w.title(), w)
print("Solución Ejercicio 1:")
print(f" - Resultado: '{clean_ex1}'")
```

Ejercicio 2:

Dada la variable `mi_edad = 30`, calcula el número de segundos aproximados que has vivido (asumiendo 365 días por año) e imprímelo en notación científica con 2 decimales usando f-strings.

```
In [ ]: # SOLUCIÓN EJERCICIO 2:
mi_edad = 30
segundos_año = 60 * 60 * 24 * 365
segundos_vividos = mi_edad * segundos_año

print("")
print("Solución Ejercicio 2:")
print(f" - Segundos vividos ({mi_edad} años): {segundos_vividos:.2e} segundos")
```



Cuestionario de Auto-evaluación y Participación

¡Pon a prueba los conocimientos adquiridos en este módulo! Completa este breve cuestionario interactivo de 5 minutos en Google Forms:

👉 **Cuestionario Módulo 01: Tipos de Datos y Fundamentos**
